

# Day 3 — Linked Lists

Faculty: Vipul Tembulwar

Doubly &amp; circular linked lists

Node storage with arrays

Stack / queue / circular queue on linked list

java.util.LinkedList internals

Yesterday's singly linked list had one weakness: it could only go forward. Today we add a backward link (doubly linked list), close the loop (circular list), and store a whole linked list inside plain arrays. Then we use linked lists to build stacks, queues and a circular queue, and look at how `java.util.LinkedList` and `LinkedHashMap` use the same ideas.

## 1. Recap — what a singly linked list cannot do well

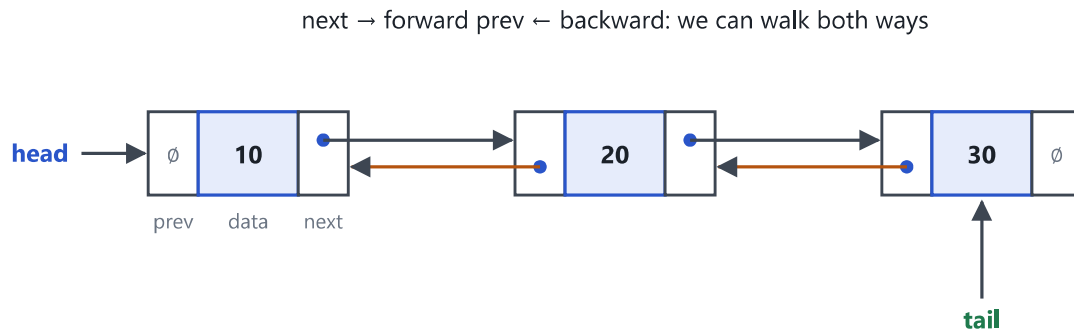
| Operation                                                                  | Singly linked<br>(head + tail)      | Problem                                                                       |
|----------------------------------------------------------------------------|-------------------------------------|-------------------------------------------------------------------------------|
| <code>addFirst</code> , <code>removeFirst</code> ,<br><code>addLast</code> | $O(1)$                              | —                                                                             |
| <code>removeLast</code>                                                    | $O(n)$                              | we must find the node <b>before</b> the tail,<br>and we can only walk forward |
| delete a node we already hold                                              | $O(n)$                              | we need its predecessor, which we<br>cannot reach from the node               |
| walk backwards (for example,<br>print in reverse)                          | $O(n)$ extra space,<br>or recursion | no backward link                                                              |

All three problems have the same fix: give every node a second reference that points **back**.

## 2. Doubly linked list

**DEFINITION** Doubly linked list

A linked list in which every node stores **data**, a reference to the **next** node, and a reference to the **previous** node.



Each node holds prev, data and next. The first node's prev and the last node's next are null.

JAVA DoublyLinkedList.java

```

private static class Node<T> {
    T data;
    Node<T> prev, next;
    Node(T data) { this.data = data; }
}

private Node<T> head, tail;
private int size;
  
```

## Adding and removing at the ends — all O(1)

JAVA

```

public void addLast(T x) {
    Node<T> n = new Node<>(x);
    if (tail == null) { head = tail = n; }           // empty list
    else { n.prev = tail; tail.next = n; tail = n; } // link both ways
    size++;
}

public T removeLast() {                               // O(1) now!
    if (tail == null) throw new NoSuchElementException("list empty");
  
```

```

T x = tail.data;
tail = tail.prev;                                // step back
if (tail == null) head = null; else tail.next = null; // cut the old last
size--;
return x;
}

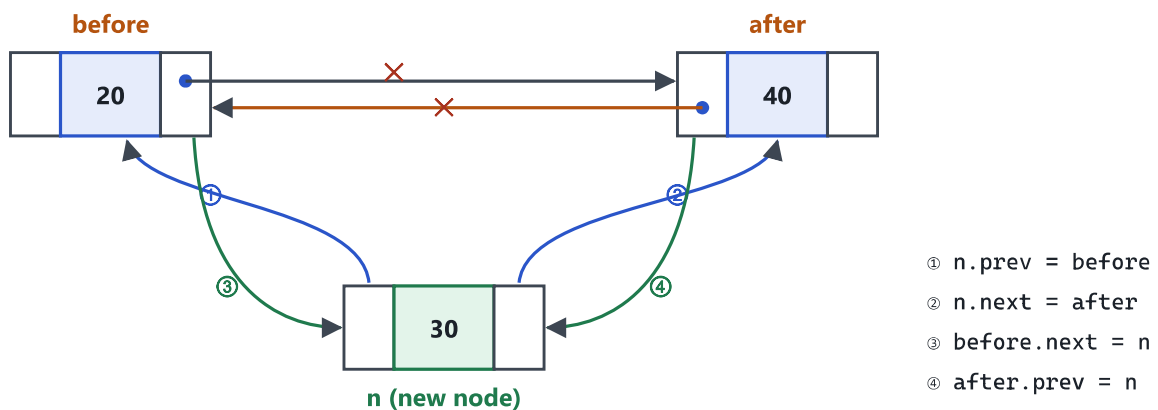
```

### WATCH OUT The two edge cases that cause most linked list bugs

1. **Empty list** — `head` and `tail` are both `null`. Adding the first node must set **both**.
2. **One-node list** — removing it must set **both** `head` and `tail` back to `null`.

Test these two cases first, every time.

## Inserting in the middle — four references



Insert 30 between 20 and 40: set the new node's two links first (①②), then redirect the neighbours (③④)

### JAVA

```

Node<T> after = nodeAt(index); // new node goes BEFORE this one
Node<T> before = after.prev;
Node<T> n = new Node<>(x);
n.prev = before; // ①
n.next = after; // ②

```

```
before.next = n;           // ③  
after.prev = n;           // ④
```

## Deleting a node we already hold — O(1)

This is the real power of a doubly linked list. If we have a reference to a node, its neighbours are right there:

JAVA

```
private T unlink(Node<T> n) {  
    if (n.prev == null) head = n.next; else n.prev.next = n.next;  
    if (n.next == null) tail = n.prev; else n.next.prev = n.prev;  
    n.prev = n.next = null;           // help the garbage collector  
    size--;  
    return n.data;  
}
```

`LinkedHashMap` (Section 7) and every **LRU cache** depend on this O(1) unlink.

## Walking from the nearer end

`get(i)` is still O(n), but we can start from whichever end is closer:

JAVA

```
private Node<T> nodeAt(int index) {  
    if (index < size / 2) {           // first half: walk forward  
        Node<T> cur = head;  
        for (int i = 0; i < index; i++) cur = cur.next;  
        return cur;  
    } else {                          // second half: walk backward  
        Node<T> cur = tail;  
        for (int i = size - 1; i > index; i--) cur = cur.prev;  
        return cur;  
    }  
}
```

```
}
}
```

Output on a 1,000-node list: `get(10)` follows 10 links, `get(990)` only **9**, and the worst case, `get(500)`, follows 499. This halves the average work, but it is still  **$O(n)$** .

#### INSIDE JAVA.UTIL Inside `java.util.LinkedList`

- It is a **doubly linked list**: `private static class Node<E> { E item; Node<E> next; Node<E> prev; }`, with fields `first`, `last` and `size`.
- `node(int index)` uses exactly the trick above: `if (index < (size >> 1))` it walks from `first`, otherwise from `last`.
- It implements **both** `List` and `Deque`, so `addFirst`, `pollLast` and `push` are all  $O(1)$ .
- Each element costs a whole `Node` object (header plus three references), roughly **4–6× the memory** of one `ArrayList` slot, and the nodes are scattered in memory, which the CPU cache dislikes.

## Singly vs doubly linked

|                            | Singly linked                 | Doubly linked                               |
|----------------------------|-------------------------------|---------------------------------------------|
| references per node        | 1 ( <code>next</code> )       | 2 ( <code>prev</code> , <code>next</code> ) |
| <code>removeLast</code>    | $O(n)$                        | <b><math>O(1)</math></b>                    |
| delete a given node        | $O(n)$ : need its predecessor | <b><math>O(1)</math></b>                    |
| walk backwards             | no                            | yes                                         |
| memory                     | less                          | one extra reference per node                |
| pointer updates per insert | 2                             | 4 (more chances for bugs)                   |

#### TIP Sentinel (dummy) nodes

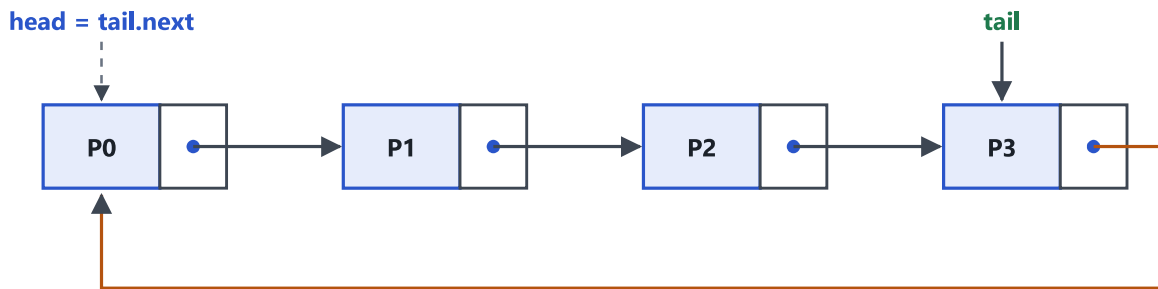
Many implementations add a dummy `header` node before the first real node and a dummy `trailer` node after the last. The list is then **never** truly empty, so the `if`

`(head == null)` special cases disappear and every insert is the same four lines. Try it in the lab.

### 3. Circular linked list

#### DEFINITION Circular linked list

A linked list in which the last node points back to the first instead of to `null`. There is no end, so you can keep going round.



last.next = first: the list has no end

addFirst: insert after tail · addLast: addFirst, then `tail = tail.next` · all  $O(1)$

Only a tail reference is stored. The head is always one step away: `tail.next`

The clever part: keep **only a tail reference**. The head is always `tail.next`, so one reference gives  $O(1)$  access to both ends.

JAVA `CircularLinkedList.java`

```
public void addFirst(T x) {                                // insert right after tail
    Node<T> n = new Node<>(x);
    if (tail == null) { n.next = n; tail = n; }           // one node points to itself
    else { n.next = tail.next; tail.next = n; }
    size++;
}

public void addLast(T x) {                                  // addFirst, then move tail forward
    addFirst(x);
```

```
tail = tail.next;
}
```

```
public void rotate() { tail = tail.next; } // the first element becomes the las
```

### WATCH OUT Traversal needs a different stopping rule

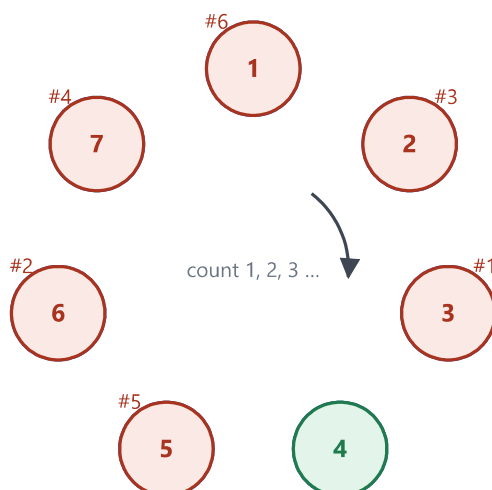
`while (cur  $\neq$  null)` never stops on a circular list, because nothing is `null` ! Use a `do ... while` loop and stop when you come **back to the starting node**:

JAVA

```
Node<T> cur = tail.next;
do {
    System.out.print(cur.data + " ");
    cur = cur.next;
} while (cur  $\neq$  tail.next);
```

### Where circular lists are used:

- **Round-robin CPU scheduling** — run the first process for one time slice, then `rotate()` . Output of `CircularLinkedList.java` : P0, P1, P2, P3, P0, P1, P2, ...
- Multiplayer turn order, music playlists on repeat, and circular buffers in networking.
- The **Josephus problem**, a classic:



```
remove #1: person 3
remove #2: person 6
remove #3: person 2
remove #4: person 7
remove #5: person 5
remove #6: person 1
```

**survivor: person 4**

$n = 7, k = 3$ . A circular list turns 'go round the circle' into following next references

JAVA `Josephus.java`

```
while (prev.next != prev) {                                // more than one person left
    for (int c = 1; c < k; c++) prev = prev.next;          // move k-1 steps
    Node out = prev.next;                                   // the k-th person
    prev.next = out.next;                                   // remove from the circle
}
```

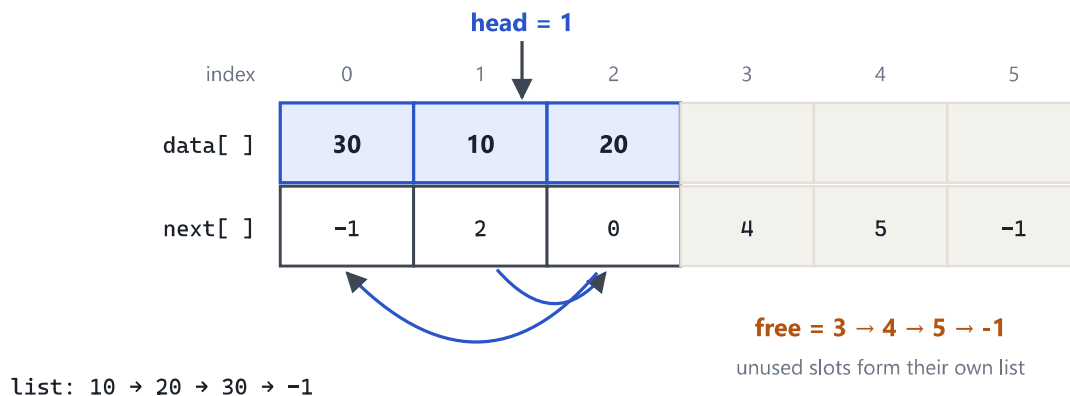
For  $n = 41$  and  $k = 3$  (the original story), the survivor is person **31**.

#### NOTE Circular doubly linked list

Combine both ideas: `last.next = first` **and** `first.prev = last`. The list with a sentinel described above is often built this way, with the single dummy node acting as both header and trailer.

## 4. Node-based storage with arrays

Can we have a linked list **without objects or references**? Yes. Store node `i` as `data[i]` plus `next[i]`, where `next[i]` holds the **index** of the next node and `-1` means null. This is sometimes called a *cursor-based* linked list.



No Node objects and no references: an index plays the role of a pointer, and -1 plays null.

After addFirst 30, addFirst 10, addAfter(10, 20). Blue = in the list, grey = on the free list.

The unused slots are themselves chained into a second list, the **free list**, so taking a slot and giving one back are both  $O(1)$ :

JAVA `ArrayLinkedList.java`

```
private int allocate() {                                // like "new Node()"
    if (free == -1) throw new IllegalStateException("no free slots");
    int slot = free;
    free = next[slot];
    return slot;
}

private void release(int slot) {                        // like the garbage collector
    next[slot] = free;
    free = slot;
}

public void addFirst(int x) {
    int s = allocate();
    data[s] = x;
    next[s] = head;                                    // exactly like n.next = head
    head = s;                                           // exactly like head = n
}
```

Output after `remove(10)` and then `addFirst(5)` shows slot 1 being **reused**:

TEXT

after `remove(10)` - slot 1 goes back to the free list:

list : 20 → 30 → null      head=2 free=1

data : [30, 10, 20, 0, 0, 0]

next : [-1, 3, 0, 4, 5, -1]

after `addFirst 5` - slot 1 is reused:

list : 5 → 20 → 30 → null      head=1 free=3

```
data : [30, 5, 20, 0, 0, 0]
```

```
next : [-1, 2, 0, 4, 5, -1]
```

## Why would anyone do this?

- **No objects to allocate or garbage-collect.** Memory pools in games, embedded systems and high-performance code preallocate arrays like this.
- **Everything sits in contiguous memory**, which is fast for the CPU cache and easy to save to a file.
- **File systems:** the FAT file system's *File Allocation Table* is literally a `next[]` array. Each cluster of a file stores the index of the file's next cluster.
- It is how linked lists were written in languages without pointers, and a common exam question.

### NOTE The same trick appears again later

Storing "links" as array indexes shows up again with the **complete binary tree in an array** (Day 5), where the children of `i` are at `2i+1` and `2i+2`, and with the **disjoint-set** arrays used by Kruskal's algorithm (Day 10).

## 5. Stack using a linked list (syllabus lab)

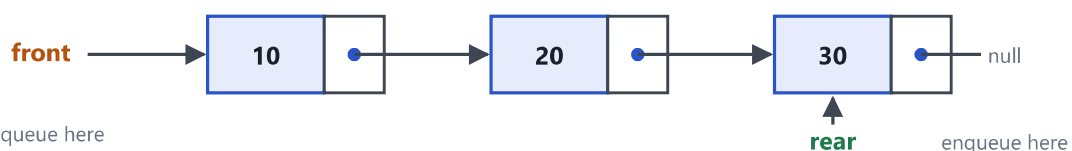
Push and pop both happen at the **head**, so both are  $O(1)$ . There is no fixed capacity, so no overflow.

### Stack



push / pop here:  $O(1)$

### Queue



dequeue here

enqueue here

Stack: everything happens at the head. Queue: add at the tail, remove at the head. Everything is  $O(1)$  and there is no fixed capacity.

JAVA `LinkedList.java`

```
public class LinkedList<T> {
    private static class Node<T> {
        T data; Node<T> next;
        Node(T data, Node<T> next) { this.data = data; this.next = next; }
    }

    private Node<T> top;           // the head of the list is the top of the stack
    private int size;

    public void push(T x) {
        top = new Node<>(x, top);    // new node points to the old top
        size++;
    }

    public T pop() {
        if (top == null) throw new NoSuchElementException("Stack underflow");
        T x = top.data;
        top = top.next;
        size--;
        return x;
    }
}
```

## 6. Queue and circular queue using linked lists

### Linear queue

Enqueue at the **tail**, dequeue at the **head**. Why not the other way round? Removing the tail of a *singly* linked list is  $O(n)$ .

JAVA `LinkedList.java`

```

public void enqueue(T x) {
    Node<T> n = new Node<>(x);
    if (rear == null) front = rear = n;
    else { rear.next = n; rear = n; }
    size++;
}

public T dequeue() {
    if (front == null) throw new NoSuchElementException("Queue underflow");
    T x = front.data;
    front = front.next;
    if (front == null) rear = null;    // queue became empty: reset rear too!
    size--;
    return x;
}

```

**QUICK CHECK** What goes wrong if we forget `if (front == null) rear = null; ?`

Show answer

## Circular queue using a linked list (syllabus lab)

Make the nodes a **ring** and keep only `rear`. The front is always `rear.next`, just like the circular list in Section 3. An optional `capacity` makes it behave like the array circular queue from Day 2, but with no `%` arithmetic.

JAVA `LinkedCircularQueue.java`

```

public void enqueue(T x) {
    if (isFull()) throw new IllegalStateException("Queue overflow");
    Node<T> n = new Node<>(x);
    if (rear == null) {
        n.next = n;    // a single node points to itself
    } else {
        n.next = rear.next;    // the new node points to the front
    }
    rear = n;
}

```

```

        rear.next = n;                // the old rear points to the new node
    }
    rear = n;                        // the new node becomes the rear
    size++;
}

public T dequeue() {
    if (rear == null) throw new NoSuchElementException("Queue underflow");
    Node<T> front = rear.next;
    if (front == rear) rear = null; // it was the only node
    else rear.next = front.next;   // bypass the old front
    size--;
    return front.data;
}

```

## TEXT

front → 10 20 30 ← rear (rear.next = 10)

dequeue → 10    front → 20 30 ← rear (rear.next = 20)

enqueue 40, 50 → front → 20 30 40 50 ← rear (rear.next = 20)    full=true

enqueue 60 → Queue overflow (capacity 4)

## Array vs linked implementations

|                    | Array (Day 2)                        | Linked list (today)                                |
|--------------------|--------------------------------------|----------------------------------------------------|
| capacity           | fixed, or resize (amortised $O(1)$ ) | grows one node at a time                           |
| every operation    | $O(1)$                               | $O(1)$                                             |
| memory per element | one slot                             | one node object (data + reference + object header) |
| memory locality    | contiguous: cache friendly           | scattered                                          |
| overflow           | possible, if fixed                   | only when the JVM is out of memory                 |
| in the JDK         | <code>ArrayDeque</code>              | <code>LinkedList</code>                            |

In practice `ArrayDeque` is usually faster. The linked version wins when you must never pay for a resize copy, or when you hold references to nodes and need to unlink them in  $O(1)$ .

## 7. Linked lists inside `java.util`

### Where `LinkedList` really wins: removing while iterating

`iterator.remove()` on a `LinkedList` just relinks the two neighbours:  $O(1)$ . On an `ArrayList` it shifts the whole tail:  $O(n)$ . Removing every even number (`ListIteratorDemo.java`):

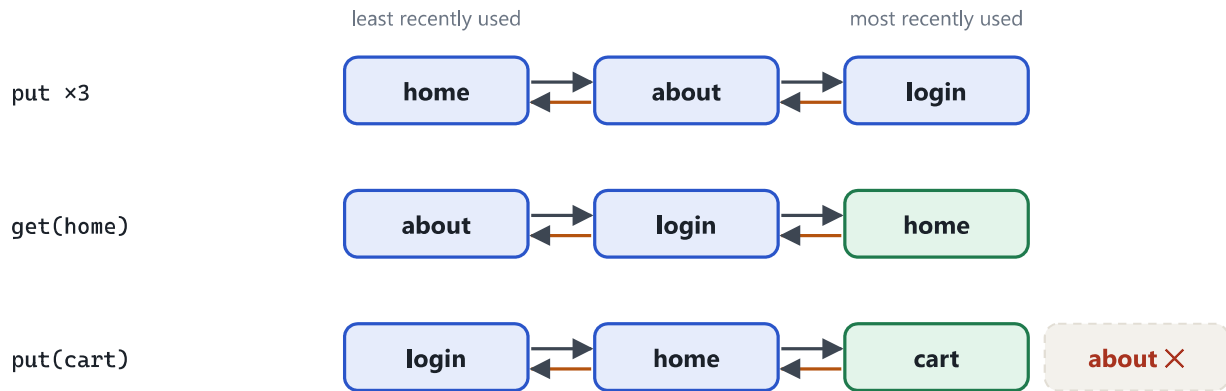
| n        | <code>ArrayList +<br/>iterator.remove()</code> | <code>LinkedList +<br/>iterator.remove()</code> | <code>ArrayList.removeIf( ... )</code> |
|----------|------------------------------------------------|-------------------------------------------------|----------------------------------------|
| 50,000   | 67 ms                                          | 8 ms                                            | 19 ms                                  |
| 1,00,000 | 230 ms                                         | 1 ms                                            | 3 ms                                   |
| 2,00,000 | 917 ms                                         | 2 ms                                            | 4 ms                                   |

#### **TIP** So which list should I use?

Use `ArrayList` by default: fast `get(i)`, compact memory, cache friendly. For bulk removal on an `ArrayList`, use `removeIf`, which compacts in one  $O(n)$  pass. Use `ArrayDeque` for stacks and queues. Choose `LinkedList` only when you really insert or remove in the middle **through an iterator**, or need a `List` and a `Deque` in one object.

### `LinkedHashMap` — a hash table threaded with a doubly linked list

`LinkedHashMap` extends `HashMap` and links **every entry** into a doubly linked list (fields `head` and `tail`, entries with `before` and `after`). That list remembers the **insertion order**. With `accessOrder = true`, every `get` moves the entry to the end instead.



The doubly linked order list makes 'move to end' and 'remove eldest'  $O(1)$

A capacity-3 LRU cache built on `LinkedHashMap(16, 0.75f, true)`

That gives an **LRU (least recently used) cache**, the kind browsers, databases and CDNs use, in a few lines:

JAVA `LRUCacheDemo.java`

```
static class LRUCache<K, V> extends LinkedHashMap<K, V> {
    private final int capacity;

    LRUCache(int capacity) {
        super(16, 0.75f, true);           // true = access order
        this.capacity = capacity;
    }

    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > capacity;         // evict the least recently used entry
    }
}
```

#### INSIDE JAVA.UTIL Why the doubly linked list matters here

The hash table finds an entry in  $O(1)$  (Day 8). The doubly linked list then lets `LinkedHashMap` **unlink it and move it to the end in  $O(1)$** , using the `unlink` idea

from Section 2. With a singly linked list, the move would be  $O(n)$ , and the cache would be too slow to be useful.

## 8. Summary

- **Doubly linked list:** `prev` + `next` .  $O(1)$  at both ends,  $O(1)$  to delete a node you already hold, and you can walk both ways. That is `java.util.LinkedList` .
- **Always test** the empty list and the one-node list. **Sentinel nodes** remove most special cases.
- **Circular list:** `last.next = first` . Keep only `tail` , since `head = tail.next` . Stop a traversal when you are back at the start. Used for round-robin scheduling and Josephus.
- **Array-based linked list:** `data[]` + `next[]` + a free list. Indexes act as pointers. Used by memory pools and the FAT file system.
- **Linked stack:** push and pop at the head. **Linked queue:** enqueue at the tail, dequeue at the head. **Linked circular queue:** a ring with a single `rear` reference.
- Prefer `ArrayList` and `ArrayDeque` in everyday Java. `LinkedList` shines with iterator-based removal. `LinkedHashMap` = hash table + doubly linked list = LRU cache.

## 9. Lab sheet — Day 3

The code files for these tasks will be shared by your faculty. Run a file directly with `java FileName.java` (Java 11+), or open it in your IDE.

### LAB Task 1 — Circular queue using a linked list (syllabus lab)

Implement `LinkedCircularQueue<T>` with a single `rear` reference and a capacity. Write a menu-driven test program. Show that `rear.next` is always the front after every operation, including when the queue becomes empty and is filled again.

### LAB Task 2 — Stack using a linked list (syllabus lab)

Implement `LinkedList<T>` . Reuse your Day 2 bracket checker and postfix evaluator with it instead of `ArrayDeque` . Then push 10 lakh elements and check that nothing overflows.

### LAB Task 3 — Doubly linked list

Implement `DoublyLinkedList<T>` with `addFirst` , `addLast` , `add(index, x)` , `removeFirst` , `removeLast` , `remove(x)` , `forward()` and `backward()` . Test the empty and one-node cases explicitly. **Bonus:** rewrite it with header and trailer sentinel nodes and count how many `if` statements disappear.

### LAB Task 4 — Circular list applications

1. Run `Josephus.java` for  $n = 10$ ,  $k = 2$  and check the answer on paper.
2. Using `CircularLinkedList` , simulate round-robin scheduling where each process needs a number of time slices, and is removed when it finishes. Print the order in which processes complete.

### LAB Task 5 — Node storage with arrays

Extend `ArrayLinkedList` with `addLast` (hint: also keep a `tail` index) and `printReverse` . After each operation, draw the `data[]` and `next[]` arrays on paper and compare them with `dump()` .

### TRY IT Home challenge

1. Detect whether a singly linked list has a **cycle**, in  $O(n)$  time and  $O(1)$  space. Hint: a slow pointer moves 1 step and a fast pointer moves 2.
2. Merge two **sorted** linked lists into one sorted list without creating new nodes.
3. Build your own `LRUCache<K, V>` with a `HashMap<K, Node>` plus your own doubly linked list, without using `LinkedHashMap` .

## 10. Quick quiz

**QUICK  
CHECK**

1. In a doubly linked list with only `head` and `tail` , what is the cost of deleting the last node?

[Show answer](#)**QUICK  
CHECK**

2. A circular singly linked list stores only `tail` . How do you reach the first node, and at what cost?

[Show answer](#)**QUICK  
CHECK**

3. In the array-based list, `head = 4` , `data = [7, 9, 3, 0, 5]` and `next = [2, -1, 1, -1, 0]` . What is the list?

[Show answer](#)**QUICK  
CHECK**

4. Which is faster in Java for `get(i)` in a loop over all elements, `ArrayList` or `LinkedList` , and why?

[Show answer](#)**QUICK  
CHECK**

5. What two structures does `LinkedHashMap` combine, and what does each one contribute to an LRU cache?

[Show answer](#)

---

**Next class — Day 4: Recursion.** Base cases, direct and indirect recursion, what happens on the call stack, pros and cons, and how to find the complexity of a recursive function.